

MediScan

Engineering Baseline Document

Codebase audit, report reconciliation, target architecture, engineering standards and roadmap for turning MediScan from a final-year project into a scalable, real-world clinical support product.

Field	Value
Version	1.0 (baseline)
Date	18 September 2026
Repository	github.com/UnavoidableVOID/MediScan (main @ 5043aa8)
Source report audited	Major_Project_Final_Report.pdf (12 Feb 2026, 68 pages)
Guiding philosophy	CUPID (Composable, Unix, Predictable, Idiomatic, Domain-based)
Status	Proposal - open decisions listed in Section 12 must be closed before Phase 1 starts

How to use this document. Section 3 is the list of what is broken today and how to fix each item - treat it as the Phase 0 backlog. Sections 5-10 define how we build from now on; every rule there is a PR review criterion. Section 11 is the roadmap. Section 12 lists decisions that are still open. Anything not in this document is not agreed.

Contents

1. Executive summary
2. Current state of the system
3. What is not working today - findings and fixes
 - 3.1 Machine learning and inference
 - 3.2 OCR and data extraction
 - 3.3 Safety layer
 - 3.4 LLM, RAG and chatbot
 - 3.5 Backend security and authorisation
 - 3.6 Backend correctness and reliability
 - 3.7 Payments and consultation
 - 3.8 Frontend
 - 3.9 Repository hygiene, secrets and privacy
4. Report vs. code - corrections needed in the academic report
5. Guiding philosophy: CUPID
6. Target architecture
7. Domain model
8. Repository layout
9. Engineering standards
10. ML lifecycle
11. Roadmap and ownership
12. Open decisions
13. Definition of done and checklists
- Appendix A - Evidence: commands and observed outputs
- Appendix B - What the project does well (keep these)

1. Executive summary

MediScan's architecture is sound and largely built: PDF upload -> OCR -> human correction -> six disease-risk models -> deterministic safety guard -> LLM summaries -> RAG chatbot -> doctor booking, payments and an admin portal. The product loop works end to end and several parts (OTP login, HttpOnly JWT cookies, doctor licence verification, trends charts, streaming chat, Dockerised ML service) are better than the final report describes.

However, a full read of the code and a live run of the deployed artifacts show that the **clinical core is not trustworthy today**:

- **Three of six deployed models are silently broken.** Heart returns a constant 67.05% for every input; Kidney returns a constant 50.89% for every creatinine ≥ 1.4 ; Thyroid has 27 classes while inference assumes 3.
- **The safety layer can hide emergencies.** Automatic OCR 'correction' divides plausible critical values by 10 (Creatinine 16 -> 1.6) before the critical-value check runs.
- **OCR evidence in the report does not hold.** The 'ground truth' column was transcribed from the OCR output, the validation set is byte-identical copies of the test set, and the one genuinely new real-world scan scores ~54%, not 95%.
- **Patients see fabricated numbers.** A hard-coded 95% 'confidence score' is displayed on every result.
- **Authorisation holes and leaked secrets.** Cross-patient data exposure in the chatbot, unrestricted writes to doctors' schedules, medical PDFs served as static files, API keys in git history, unredacted patient reports committed to a public repository.

None of this is visible from the UI - every failure degrades into plausible-looking output. That is the core lesson for the rebuild: **the system must fail loudly and predictably**. The plan below adopts CUPID as the guiding philosophy, rebuilds the inference and training layers, restructures (not rewrites) the Django backend around a proper clinical domain model, and introduces the foundations - contracts, async jobs, object storage, policy-based authorisation, audit logging, CI with the tests that would have caught every finding above - before adding features.

Estimated effort: 12-14 weeks for four people working in parallel by domain. Phase 0 (one week) stops the bleeding on the current code and can start immediately.

2. Current state of the system

2.1 Stack as deployed

Layer	Technology	Notes
Frontend	React 19, Vite 7, Redux Toolkit, Tailwind 4, Recharts, framer-motion (JavaScript)	~11.4k lines in frontend/src; no TypeScript; hand-written axios/fetch calls
API	Django 4.2 + DRF, SimpleJWT (HttpOnly cookies), drf-spectacular, Google OAuth, email OTP	Apps: authentication, reports, doctor, admin_panel, chatbot. SQLite in use (psycopg installed, unused)
ML service	FastAPI, scikit-learn, XGBoost, Tesseract + pdf2image + OpenCV, LangChain, FAISS, sentence-transformers	Port 8001, no authentication, CORS '*'. Docker multi-stage image exists
LLM	Groq llama-3.1-8b-instant (cloud) by default; Ollama llama3.2 fallback	USE_CLOUD_LLM=True in ML/.env
Payments	Khalti (two different API versions in code)	Sandbox key hard-coded
Infra	Local dev only; Vercel origin allowed in CORS	No CI, no tests beyond a 99-line auth test

2.2 Data flow today

```
Browser -> POST /api/reports/ (Django saves PDF to Backend/media/reports/)
        -> POST /api/reports/{id}/process/
            Django -> POST ML:8001/extract_from_pdf (pdf2image -> Tesseract --psm 6 -> substring keyword match)
            ML applies SafetyGuard.sanitize_data() (auto /10 "fixes") -> extracted_data
        -> user edits values in UI
        -> PUT /api/reports/{id}/correct/
            Django -> POST ML:8001/analyze_verified_data
            SafetyGuard.check_criticals()
            DiseasePredictor.analyze_full_report() (6 models, triggered by presence of a key)
            MedicalReportGenerator (patient + doctor LLM summaries)
            Django stores ReportResult (summary, conditions, risk_level, confidence_score=95.0)
        -> GET /api/reports/{id}/result/
Chat: Browser -> POST /api/chatbot/chat/ -> ML:8001/chat_stream (RAG, k=3, no history)
```

2.3 What genuinely works

- Two-step OCR with human verification (raw_ocr_data / final_data / is_corrected) - the right design for medical OCR.
- Diabetes ensemble (XGB+RF+SVM, SMOTE, threshold tuning) and Liver voting classifier are trained and loaded correctly.
- Anemia model: features align with OCR output; the only model that is fully consistent end to end.
- Role-aware summaries (patient vs doctor) with serializer-level hiding of the doctor summary from patients.
- Specialist routing from detected conditions into doctor search and booking.
- Authentication: OTP on every login, HttpOnly JWT cookies, OAuth blocked for doctors, licence verification state machine.
- Longitudinal trends endpoint and chart; streaming chat; revenue split and refund policy; Dockerised ML service.

3. What is not working today - findings and fixes

Severity scale: **S1** clinically or legally dangerous / data exposure; **S2** produces wrong results for users; **S3** reliability or maintainability; **S4** hygiene. Every S1/S2 item is Phase 0 or Phase 2 work.

3.1 Machine learning and inference

#	Sev	Finding	Evidence	Fix
ML-1	S1	Heart model outputs a constant. Inference loads heart_model_columns.pkl (Dec 14 order) but model/scaler were retrained Dec 31 with a different column order. 15 columns either way, so no error - features are scaled with the wrong statistics.	inference.py:34,117. Chol=120 and Chol=350 both -> 'Heart Disease 67.05%'. Scaler mean for the column inference calls 'Sex' is 132.4 (RestingBP).	Model bundle with manifest listing feature names; assert scaler.feature_names_in_ == manifest features at load. Sensitivity test in CI. Retrain from one pipeline.
ML-2	S1	Kidney model outputs a constant. kidney_columns.pkl = ['sc','bu','hemo',...]; OCR emits 'Creatinine','Blood_Urea','Hemoglobin'. reindex() turns all inputs into NaN -> imputer fills medians -> same prediction always.	inference.py:41,91. Cr 1.5/3.0/4.9 all -> 'CKD 50.89%'. Only the hard-coded 'creatinine < 1.4 -> Healthy' rule works.	Canonical analyte codes (LOINC) end to end; manifest-driven feature mapping; refuse (NotAssessable) when required features are missing instead of imputing.
ML-3	S1	Kidney model is LogisticRegression, not Random Forest as Table I states. train_kidney.py keeps the first model on tied F1.	type(joblib.load('kidney_best_model.pkl')) == LogisticRegression	Selection rule must be explicit and recorded in manifest; report regenerated from manifests.
ML-4	S1	Thyroid model has 27 classes; inference assumes 3. Deployed.pkl came from universal_trainer.py which label-encoded raw Garvan diagnosis codes ('-', 'A', 'AK', ..., 'GK'). Any class >= 3 becomes 'Unknown'; probs[pred] indexes by label not position. patient_id is a training feature.	classes_ = [0,1,2,3,4,5,6,9,...,31]; thyroid_columns.pkl contains 'patient_id', 'TBG', '*_measured_t'.	Map codes to {euthyroid, hyper, hypo} deliberately in the pipeline; drop id/leak columns; macro-F1; manifest declares class labels.
ML-5	S2	Unit/semantic mismatch. OCR maps 'Free T4' -> TT4 (total T4, ~60-150 nmol/L) and 'Free T3' -> T3. Report1's FT4 1.19 ng/dL is fed as total T4. PIMA 'Glucose' is 2-hour OGTT; fasting glucose is fed instead.	ocr_main.py:46-47. Fasting glucose 100 -> 'Diabetic 35.7%' at threshold 0.20.	Analyte registry with canonical units + conversion; models declare the exact analyte they need; Diabetes reframed (see 10.4).
ML-6	S2	Threshold drift. Diabetes ensemble tuned/reported at 0.30, deployed at 0.20.	train_diabetes_ensemble.py:56, audit_models.py:45 vs inference.py:65	Threshold lives in the manifest only; inference reads it.
ML-7	S2	Heart hard-codes 9 of 11 features (ChestPainType='NAP', MaxHR=150, ST_Slope='Up' ...) because a blood report has none of them. 172 training rows have Cholesterol=0, never cleaned.	inference.py:105-109; heart.csv	Either retrain on blood-panel-available features (age, sex, BP, cholesterol, fasting glucose) or remove Heart from v1. Decision D4.
ML-8	S2	Anemia never affects risk level. predict_anemia returns no risk_score, so max_risk_score ignores it.	inference.py:200; analysis_service.py:59-63	Uniform Assessment type: every model returns probability + label + threshold.
ML-9	S3	Silent fallbacks. except: df.fillna(0); reindex without validation; warnings suppressed globally.	inference.py:7,48-51	Predictable: validate with pydantic, raise, log with context. No bare except.
ML-10	S3	Six competing training scripts write to the same.pkl names; no record of which produced the deployed artifact. No CV; train_thyroid.py has no test split at all.	ML/src/training/*.py; models/*.pkl timestamps	One pipeline (ml/pipelines/train.py) -> versioned bundle + eval report + model card.

3.2 OCR and data extraction

#	Sev	Finding	Evidence	Fix
OCR-1	S1	Wrong values on real scans that would trigger false emergencies. On Report4 (Civil Service Hospital, the only real out-of-distribution layout) Creatinine 0.9 is read as 9.0 (critical limit 5.0 -> 'Severe Renal Failure' alert on a healthy patient).	Run of parse_lab_report on Report4: Creatinine 9.0; Hemoglobin, WBC, RBC, Sodium missed; 'ALT 4.0' hallucinated (not on the report). 7/13 fields correct.	Row-based parsing from Tesseract TSV (name value unit range on one line), value must be adjacent to unit or inside the printed range x 10; confidence per field; low-confidence fields highlighted in review UI.
OCR-2	S2	Substring keyword matching. 'MCH' matches the 'MCHC' line first -> MCH reported 34 (true 29) on Report1. 'T3', 'Hb', 'ALT' match unrelated words.	ocr_main.py:107 keyword.lower() in line.lower()	Word-boundary regex per alias; longest alias wins; aliases table in packages/clinical.
OCR-3	S2	Value fitted to a test file. if str(int(value)).startswith('70'): return 70.0 exists to make Report1's glucose pass.	ocr_main.py:93-94	Delete. Golden-set test decides correctness, not special cases.
OCR-4	S2	Image uploads accepted then silently fail. Serializer and UI accept JPG/PNG; ML calls pdf2image only -> returns [] -> 'Could not read data'. UI error text says 'Only PNG files are supported' (wrong twice).	reports/serializers.py:55; CheckReports.jsx:106-110; ocr_main.py:18	Accept PDF + images; branch on MIME; convert images with PIL.
OCR-5	S2	Preprocessing is minimal. Otsu threshold only; no deskew, no DPI normalisation, --psm 6 for every page.	ocr_main.py:9-14	Deskew (Hough/minAreaRect), upscale to 300 DPI, adaptive threshold for photos, try psm 4/6 and keep the higher-confidence result.
OCR-6	S3	No provenance. Extracted values carry no page/bbox/confidence, so the review UI cannot show where a number came from.	extract_text_with_layout returns list[str]	Observation.bbox + ocr_confidence; UI highlights the region on the page image.
OCR-7	S3	Report generator does not produce what the report claims (fpdf, 4 files, no noise/skew/fonts vs 'ReportLab/Faker, 500+, blur/skew').	generation/generate_lab_pdfs.py	Synthetic generator with layout templates from real Nepali labs, random fonts, skew, blur, JPEG artefacts; used only for stress tests, never as 'validation'.

3.3 Safety layer

#	Sev	Finding	Evidence	Fix
SAF-1	S1	Auto-correction masks true critical values. sanitize_data divides by 10 above a threshold, before check_criticals runs.	Creatinine 16.0 -> 1.6 (alert suppressed); Bilirubin 25.0 -> 2.5 (suppressed); TSH 150 -> 15.	Corrections become Suggestions shown in the review UI (accept/reject). Safety rules run on the human-verified values only and never mutate.
SAF-2	S1	Critical alerts are dropped. Django reads critical_alerts only to set risk_level='High'; nothing persists or renders them. The 'Panic Button' survives as the string 'URGENT:' inside LLM prose.	analysis_service.py:47,66; grep critical frontend/src -> no results	CriticalFlag model; rendered as a red banner before any summary; included in doctor notifications.
SAF-3	S2	Rules are Python literals with no source, no tests, and thresholds that mix adult/paediatric and sex-specific ranges (Hemoglobin min 6 for everyone).	safety.py:14-23	Rules as YAML with guideline source; sex/age-aware where the guideline is; positive and negative test per rule.
SAF-4	S3	SafetyGuard prints to stdout and returns mutated copies - not composable or observable.	safety.py:39	Pure function: evaluate(observations) -> list[CriticalFlag]; structured logging by caller.

3.4 LLM, RAG and chatbot

#	Sev	Finding	Evidence	Fix
LLM-1	S1	Cross-patient data exposure (IDOR). ReportResult is fetched by report_id without a user filter; any logged-in user can pass another patient's report_id and receive their AI summary and conditions as chat context.	chatbot/views.py:28	Policy check can_view_report(user, report) before building context; test in policy matrix.
LLM-2	S2	No conversation memory. chatbot/models.py is empty; every message is stateless. Prompt hard-codes 'I have analyzed your report' even when no report was passed.	rag.py:127; chatbot/models.py	Conversation + Message models; last N turns in prompt; greeting conditional on context.
LLM-3	S2	No output validation. Nothing checks that numbers in generated text exist in the input - the 'generative consistency' check promised in report section 6 was never built.	report_generator.py, rag.py	Validator: every numeric token must appear in observations or retrieved chunks; otherwise regenerate once, then redact.
LLM-4	S2	Deployment story is inconsistent. Report says fully offline; .env says USE_CLOUD_LLM=True with Groq. report_generator.py hard-codes localhost:11434 while rag.py reads OLLAMA_BASE_URL (breaks inside Docker).	ML/.env; report_generator.py:38	Single LLM gateway with provider adapter and one config; de-identified payload (values only) if cloud is used. Decision D5.
LLM-5	S3	Knowledge base differs from the report (ACG/ATA/ESC/KDIGO/WHO PDFs + 26-line txt vs 'WHO, CDC, PubMed'). Retrieval returns no citations; chunk 1000/overlap 100 untuned; FAISS index built once, not reproducible.	ML/knowledge_base/; ingest.py	Versioned corpus manifest; citations (doc, page) returned with every answer; ingestion is a CI job.
LLM-6	S3	Freemium gate (is_premium) is dead code - Django always sends True.	main.py:145; chatbot/views.py:63	Remove, or implement entitlements properly in the billing domain.
LLM-7	S3	Django -> ML requests have no timeout; a slow LLM blocks a Django worker indefinitely.	analysis_service.py:40; ocr_service.py:24	All inference/LLM work runs in the Celery worker with per-step timeouts and retries.

3.5 Backend security and authorisation

#	Sev	Finding	Evidence	Fix
SEC-1	S1	Any authenticated user can create, update or delete any doctor's availability. Write actions return <code>IsAuthenticated</code> ; with <code>?doctor=</code> the queryset exposes another doctor's slots.	<code>doctor/views.py:289-295, 297-313</code>	Policy: <code>can_manage_availability(user, doctor) = user is that doctor or is_staff</code> ; object-level permission on every write.
SEC-2	S1	Uploaded medical PDFs are publicly served. <code>MEDIA_URL</code> via <code>static()</code> under <code>DEBUG=True</code> (default <code>True</code>), <code>ALLOWED_HOSTS=[*]</code> .	<code>core/urls.py:56-58</code> ; <code>settings.py:49-51</code>	Object storage with short-lived signed URLs issued only after policy check; <code>DEBUG</code> default <code>False</code> .
SEC-3	S1	Inference service has no authentication and <code>CORS '*'</code> . Anyone reaching port 8001 can run inference or the LLM.	<code>ML/src/main.py:22-34</code>	Service-to-service auth (shared secret or mTLS); bind to internal network only; drop <code>CORS</code> .
SEC-4	S1	Secrets in git. Groq API key in <code>ML/README.md</code> history (commits <code>4df8c74</code> , <code>bd83850</code>) and in <code>ML/test_pipeline.txt</code> on disk; Khalti secret key hard-coded in <code>doctor/services.py:13</code> ; Django <code>SECRET_KEY</code> literal in <code>settings.py:41</code> .	<code>git log -S'gsk_'</code>	Rotate all three now; gitleaks pre-commit + CI; <code>.env.example</code> only.
SEC-5	S1	Unredacted patient documents committed to a public repo. <code>Backend/reports/Report1.pdf</code> shows a team member's name, age, hospital and sample numbers; <code>Report4 (local)</code> shows another person's name, address, patient ID. Report claims 'All PII manually redacted'.	<code>git ls-files grep pdf</code>	<code>git filter-repo</code> to purge; redact; keep golden set in private storage or with consent forms.
SEC-6	S2	Doctors can read any patient's trends - the linkage check is a literal <code>'# Optional'</code> comment.	<code>reports/views.py:148</code>	Policy check; tested.
SEC-7	S2	Account lockout fields (<code>failed_login_attempts</code> , <code>account_locked_until</code>) exist but nothing sets them; no rate limiting on OTP or login.	<code>authentication/models.py:84-85</code>	<code>django-ratelimit</code> or DRF throttles on auth endpoints; lockout implemented or fields removed.
SEC-8	S2	No audit trail of who viewed/downloaded which report.	-	audit app: <code>AccessEvent(actor, action, object, ip, ts)</code> written from policies.
SEC-9	S3	Privacy Policy route is a placeholder in an app whose report cites GDPR/HIPAA.	<code>App.jsx:147</code>	Write it; add consent capture at upload; data retention/deletion endpoint.

3.6 Backend correctness and reliability

#	Sev	Finding	Evidence	Fix
BE-1	S2	Fabricated confidence. confidence_score is hard-coded 95.0 and rendered as an animated bar.	analysis_service.py:82; ViewReportResult.jsx:264-269	Remove, or show per-assessment probability with plain-language calibration note.
BE-2	S2	SQLite in production settings while the report claims PostgreSQL; no migrations strategy, no backups.	settings.py:108	PostgreSQL everywhere (compose in dev); daily backups; migration check in CI.
BE-3	S2	Analysis runs synchronously inside the HTTP request (OCR + 6 models + 2 LLM calls). No queue, no retry, no status.	reports/views.py:106	Celery job; ReportJob(status, step, error); UI polls or subscribes.
BE-4	S3	Business logic in viewsets and print('DEBUG') statements in request paths.	reports/views.py, doctor/views.py, ocr_service.py	Service layer per domain; structured logging with request/job IDs.
BE-5	S3	DoctorPatientLink.patient is OneToOne and LinkDoctorView deletes the previous link - no history across doctors.	doctor/models.py:10; doctor/views.py:72-74	Many-to-many with status and dates (Consultation episodes).
BE-6	S3	Two definitions of SECRET_KEY; DEBUG defaults to True; drf_spectacular ENUM overrides reference stale paths.	settings.py:41,47,49	Single settings module with env-driven values; fail fast if SECRET_KEY missing.
BE-7	S4	Test coverage: 99 lines (auth) + three 3-line stubs. Nothing for OCR, models, safety, payments, permissions.	*/tests.py	See Section 9.4.

3.7 Payments and consultation

#	Sev	Finding	Evidence	Fix
PAY-1	S2	Two incompatible Khalti flows: ePayment v2 (pidx) in khalti_views.py and legacy v1 (token) in AppointmentViewSet.verify_payment.	doctor/khalti_views.py; doctor/views.py:537-600	One flow (ePayment v2) with server-side lookup + webhook; remove the other.
PAY-2	S2	Appointments deleted on transient failures. Init failure or any non-Completed lookup status deletes the appointment; a Khalti timeout wipes the booking.	khalti_views.py:41,91	State machine: PENDING_PAYMENT -> PAID PAYMENT_FAILED EXPIRED; never delete; idempotent verify keyed on pidx.
PAY-3	S2	'Refund initiated' is a DB field; no refund API call is made.	doctor/views.py:501-516	Real refund via gateway or clearly labelled manual refund queue for admin.
PAY-4	S2	Booking does not validate the slot exists in DoctorAvailability, or that the doctor is VERIFIED, or that the slot is in the future.	doctor/views.py:419-448	Slot validation in consultation service; unique constraint (doctor, date, start_time) where status in PENDING/PAID.
PAY-5	S3	Race window between create and verify handled by a read-then-write check, not a DB constraint.	doctor/views.py:550-559	Partial unique index + select_for_update.

3.8 Frontend

#	Sev	Finding	Evidence	Fix
FE-1	S2	Critical alerts, 'not assessed' states and OCR confidence are not representable in the UI; results page shows summary prose and a fake confidence bar.	ViewReportResult.jsx	Result page redesign: critical banner -> per-condition cards (assessed / not assessed / probability) -> summary -> doctor comment.
FE-2	S3	No TypeScript; API paths hand-written in slices; env fallback ports inconsistent (8000 vs 8001 across docs).	store/slices/*.js; services/api.js	TypeScript + client generated from OpenAPI; single VITE_API_URL.
FE-3	S3	Report result fetched by id with no job status - if analysis is slow the page shows 'not ready' with no progress.	reportsSlice.js	Job status endpoint + polling/SSE; step-wise progress (extracting -> checking safety -> assessing -> writing summary).
FE-4	S4	Placeholder routes (demo, privacy), root README is the Vite template, misleading validation messages.	App.jsx:144-147; README.md	Remove or implement; real README.

3.9 Repository hygiene, secrets and privacy

#	Sev	Finding	Fix
REP-1	S1	Real patient PDFs and a doctor licence PDF tracked in git (Backend/reports/*.pdf, Backend/doctor_licenses/license.pdf); Groq key in history.	Purge with git filter-repo, force-push, rotate keys, notify collaborators to re-clone.
REP-2	S3	Model pickles (up to 84 MB thyroid) and FAISS index committed to git.	Object storage / Git LFS; bundles pulled by version at start-up.
REP-3	S3	Unrelated project (Booking/ - elephant safari) inside the tree; conflicts.txt, status_output.txt, create_doctor.py (password123) at root; duplicate SampleReport_*.pdf files.	Delete; seed data via a management command reading env.
REP-4	S4	No CI, no pre-commit, no linting config, no .env.example, no ADRs.	Section 9.

4. Report vs. code - corrections needed in the academic report

These are statements in Major_Project_Final_Report.pdf that the code contradicts. Each must be corrected before the report is reused (viva, publication, portfolio). Where the code is right and the report undersells it, that is noted too.

Report states	Code shows	Correction
PostgreSQL (section 3.2)	SQLite (settings.py:108)	Switch to PostgreSQL (BE-2) and then the report is true.
'Keeping all processing offline' / local Llama 3.2 (4.3, 3.3.3)	USE_CLOUD_LLM=True, Groq. Conclusion (6) admits Groq.	State the actual deployment mode once; describe de-identification if cloud.
Knowledge base: WHO, CDC fact sheets, PubMed abstracts	ACG, ATA, ESC, KDIGO, WHO x2 PDFs + 26-line txt	List the real corpus with versions.
'All PII was manually redacted' from 20 local reports	4 unique real PDFs; two with full names, IDs, addresses; committed publicly	Redact, purge, and state the true count.
500+ synthetic reports (ReportLab/Faker) with fonts, blur, skew	fpdf, 4 files, no noise	Build it (OCR-7) or remove the claim.
OCR validation on Report1, 4, 7, 9, 12	Report5/10 = Report1, 6 = 2, 7/9 = 3, 8 = 4 (md5). Report12 does not exist.	New golden set with distinct documents.
OCR 'Ground Truth (Real)' column	Matches OCR output, not the document: Total Protein 6.9 vs 6.0, Sodium 135.52 vs 135.0, Potassium 4.40 vs 4.0, MCH 29 vs 34	Hand-transcribe truth; report the real EMR (Report4 ~54%).
Kidney: Random Forest, Recall 100%	LogisticRegression deployed; constant output in production	Regenerate Table I from bundle manifests.
Thyroid: Normal / Hyper / Hypo, F1 93.2%	27 label-encoded diagnosis codes; weighted F1 dominated by the 74% negative class	Deliberate 3-class mapping; macro-F1 with CV.
Heart: Cleveland UCI, 14 attributes	Kaggle 'Heart Failure Prediction' (918 rows, 12 cols); 172 rows Cholesterol=0	Cite the real dataset; clean or drop zeros.
Heart table: SVM 0.90/0.89/0.941/0.914 vs others ~0.89	Four rows are weighted averages from benchmark_results.csv (recall == accuracy); SVM row is binary recall from another run	One metric definition per table; regenerate.
'Our Neural Network experiment failed to beat RF on Diabetes' (2.2)	MLP exists only in train_liver.py	Remove or run the experiment.
'Spatial analysis algorithms' for tables	Line-by-line substring matching	Describe the actual method, or build OCR-1.
Explainable AI layer (abstract, keywords)	No SHAP/LIME/feature importance; explanation = LLM prose	Rename to 'LLM-generated explanations' or add SHAP per assessment.
k-fold cross-validation (6)	Only GridSearchCV cv=3 in one script; tables are single 80/20 splits; thyroid has no split	Section 10 evaluation protocol.
Use case: Lab technicians, bulk upload, subscriptions, emergency alerts, high-risk leads	None implemented; is_premium is dead code	Remove from diagrams; add what exists (trends, OAuth, OTP, streaming chat, revenue).
Panic Button described as key safety feature	No evaluation in section 5	Add SafetyGuard evaluation with synthetic critical cases.
Batman test 'refused to answer'	Screenshot offers 'I can provide general information about Batman'	Re-run with current prompt; report honestly.
Docker as future work (7)	Multi-stage Dockerfile already exists	Move to deliverables.
Two sections numbered 6; future tense in a final report; figure numbering drift; 2021-proposal timeline	-	Editorial pass.

5. Guiding philosophy: CUPID

We adopt **CUPID** (Dan North) over SOLID. SOLID is five rules about class design in object-oriented code. CUPID is five *properties* that code should have in any language. MediScan is polyglot (Python/Django, Python/FastAPI, TypeScript/React, sklearn artifacts, YAML rules) and almost none of the defects in Section 3 were class-design problems - they were property failures. CUPID names them directly.

Property	Definition	What it forbids in MediScan	What it requires
Composable	Small surface area, minimal dependencies, intention-revealing; plays well with others	One function doing safety + 6 models + 2 LLM calls; SafetyGuard printing and mutating; models loaded as dicts of pickle files	Pure functions in packages/clinical; every pipeline step has one input type and one output type; services communicate via typed contracts
Unix philosophy	Does one thing well; composes via simple interfaces	ocr_main.py OCRs, parses, maps features and patches glucose; views that call external services and compute risk	Steps: extract -> normalise -> review -> evaluate rules -> assess -> explain, orchestrated by a worker, each independently testable
Predictable	Does what it looks like; deterministic; observable	Silent reindex to NaN; fillna(0); except: pass; hard-coded 95%; auto /10 corrections; constants returned as predictions	Validate inputs, raise on contract violations, NotAssessable instead of guessing, structured logs and metrics, idempotent jobs, sensitivity tests
Idiomatic	Feels natural to someone who knows the language and framework	print('DEBUG') in views; requests without timeouts; business logic in viewsets; hand-written fetch URLs	Django apps + services + policies; FastAPI + pydantic; React feature folders + TanStack Query + generated client; ruff, mypy, eslint, tsc
Domain-based	Code reads in the language of the problem domain	ExtractedReportData.final_data JSON blob; 'Total_Protiens'; dict keys as the data model	Observation, Analyte, ReferenceRange, CriticalFlag, RiskAssessment, Consultation, Payment - with LOINC codes and canonical units

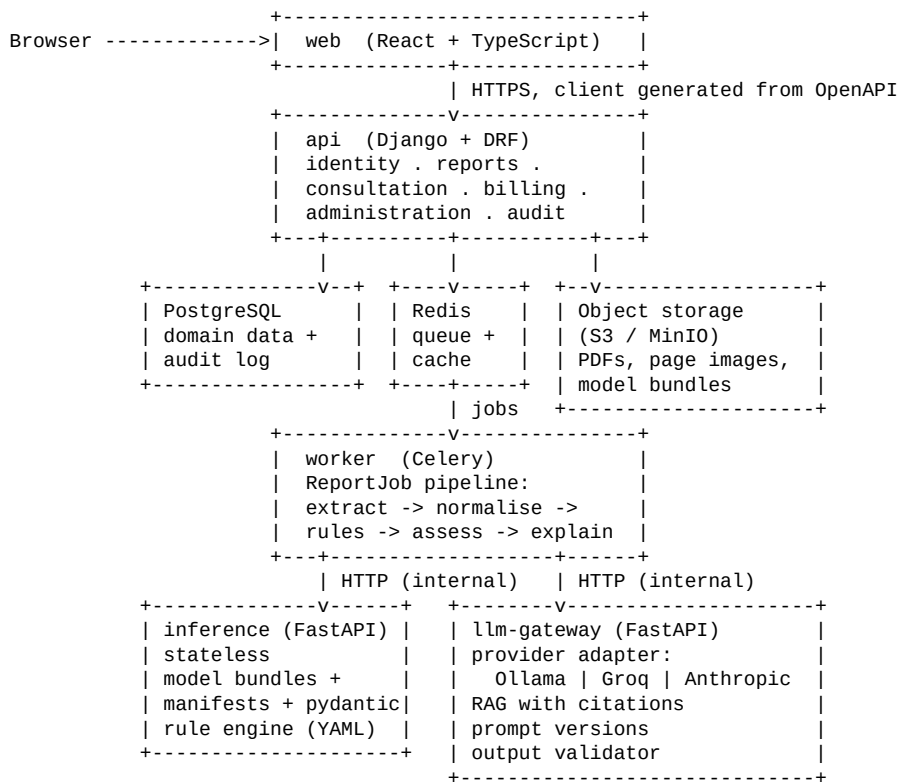
Relationship to SOLID. We will still use dependency inversion where a swap point is needed (LLM provider, storage backend, payment gateway, OCR engine). CUPID's Composable property gets us there without abstract factories. The review question for every PR is: *can a new team member read this, predict what it does, and test it in isolation without running Docker?*

6. Target architecture

6.1 Rewrite or restructure - decision per layer

Layer	Today	Decision	Rationale
Inference service	~1,000 lines	Rewrite	Its core abstractions (dict in/out, loose pickle files) caused ML-1..ML-9. Cheaper to rebuild than untangle.
Training code	12 scripts, ~1,200 lines	Rewrite as one pipeline	Scripts disagree with each other; no provenance.
Django API	~4,000 lines	Keep, restructure	Auth, OTP, licence workflow, admin are sound. Add domain apps, service layer, policies; rewrite reports + chatbot apps.
React web	~11,400 lines	Keep, migrate incrementally	UI is fine; move to TypeScript + generated client feature by feature.

6.2 System diagram



6.3 Key structural decisions

a. Report analysis is an asynchronous job

Upload returns a job id immediately. The Celery worker runs the pipeline step by step, recording status and the failing step on error. The UI polls (v1) or subscribes via SSE (v2). Each step has its own timeout and retry policy. This removes BE-3, LLM-7 and is the scalability primitive: throughput scales by adding workers, and the inference and LLM services scale independently.

b. A canonical clinical data model

Extraction produces **Observations**, not dict keys. Each Observation references an **Analyte** from a registry keyed by LOINC code with a canonical unit and known aliases (English, common Nepali lab spellings). Unit conversion is an explicit step. The lab's own printed reference range is preserved. This removes ML-2, ML-5, OCR-2 and gives HL7 FHIR compatibility for the HMS integration in the report's future work.

c. Model bundles with contracts

A deployed model is a directory: model file + **manifest.json** declaring required and optional analytes (LOINC + unit), class labels, threshold, metrics, dataset hash, git SHA, model card. The inference service validates every request against the manifest and returns **NotAssessable(reason)** when a required analyte is missing - never imputes a median for a real patient. Load-time assertion: `scaler.feature_names_in_ == manifest.features`. This removes ML-1, ML-3, ML-4, ML-6, ML-7.

d. Safety is a rule engine that emits flags, never edits data

Rules are YAML (analyte, comparator, value, sex/age scope, severity, message, guideline source). OCR corrections become **Suggestions** in the review UI. Rules run on human-verified Observations, emit **CriticalFlags** that are persisted, rendered first, and injected into the LLM prompt as hard constraints. Removes SAF-1..SAF-4.

e. LLM gateway: adapter, grounding, validation

One service owns every model call. Provider selected by config (Ollama for private/local, Groq or Anthropic for cloud with a de-identified payload). RAG returns chunk citations with each answer. An output validator checks that every number in generated text exists in the observations or retrieved text. Prompts are versioned files. Conversations are persisted with the report they refer to. Removes LLM-2..LLM-6.

f. Authorisation as policy functions

Each domain app has `policies.py`: `can_view_report(user, report)`, `can_manage_availability(user, doctor)`, etc. Every view calls a policy; every policy has a row in the role x resource x action test matrix. Removes SEC-1, SEC-6, LLM-1.

g. Audit log and signed URLs

Every read or download of PHI writes an `AccessEvent`. Files live in object storage and are served only through short-lived signed URLs issued after a policy check. Removes SEC-2, SEC-8.

h. Scalability path (do not over-engineer)

v1: one VPS with Docker Compose (api, worker, inference, llm-gateway, postgres, redis, minio, caddy). All services stateless except the databases, so the path to horizontal scaling is: more workers -> more inference replicas -> managed Postgres -> Kubernetes only when metrics show the need. GPU is not required; Ollama on CPU or a cloud provider covers v1.

7. Domain model

Names below are the vocabulary for code, database, API and conversation. If a concept is not here, add it here first.

Analyte	loinc_code, name, canonical_unit, aliases[], category (CBC LFT RFT TFT LIPID GLUCOSE)
Observation	report, analyte, value, unit, value_canonical, ref_low, ref_high (as printed), source (OCR MANUAL), ocr_confidence, page, bbox, created_by
Suggestion	observation, proposed_value, reason (DECIMAL_SHIFT UNIT_MISMATCH OUT_OF_RANGE), accepted (bool null)
CriticalFlag	report, analyte, value, rule_id, severity (CRITICAL_HIGH CRITICAL_LOW), message, guideline_ref
RiskAssessment	report, condition (DIABETES CKD LIVER THYROID ANEMIA HEART), model_version, status (ASSESSED NOT_ASSESSABLE), probability, label, threshold, missing_analytes[]
Explanation	report, audience (PATIENT CLINICIAN), text, prompt_version, provider, citations[], validated (bool)
ReportJob	report, status (QUEUED EXTRACTING AWAITING_REVIEW EVALUATING ASSESSING EXPLAINING DONE FAILED), step_errors, started_at, finished_at
Report	patient, file_key (object storage), uploaded_at, lab_name, collected_at, job
Conversation	patient, report (nullable), messages[] (role, text, citations, created_at)
Consultation	patient, doctor, status (REQUESTED PAID COMPLETED CANCELLED EXPIRED), slot, payment
AvailabilitySlot	doctor, date, start, end, label, is_active
Payment	consultation, gateway (KHALTI), external_id (pidx), amount, status, raw_payload, verified_at
LicenceReview	doctor, documents[], status, reviewer, decision_at, reason
AccessEvent	actor, action (VIEW DOWNLOAD EDIT EXPORT), object_type, object_id, ip, at

Rules for the model: Observations are append-only (a correction creates a new Observation with source=MANUAL and supersedes the OCR one); RiskAssessments are immutable per model_version; Explanations always reference the RiskAssessments and CriticalFlags they were generated from.

7.1 Analyte registry - initial scope

Panel	Analytes (LOINC)	Notes
CBC	Hemoglobin 718-7, WBC 6690-2, RBC 789-8, Platelets 777-3, MCV 787-2, MCH 785-6, MCHC 786-4, PCV/Hct 4544-3	Nepali labs print 'Total Leucocyte Count', 'Total RBC Count', 'Haemoglobin' - aliases required
LFT	Total bilirubin 1975-2, Direct bilirubin 1968-7, ALT/SGPT 1742-6, AST/SGOT 1920-8, ALP 6768-6, Total protein 2885-2, Albumin 1751-7	A/G ratio derivable
RFT	Creatinine 2160-0, Urea 3091-6, Sodium 2951-2, Potassium 2823-3, Uric acid 3084-1	Urea vs BUN conversion (x 0.467)
TFT	TSH 3016-3, Free T4 3024-7, Total T4 3026-2, Free T3 3051-0, Total T3 3053-6	Free vs Total are different analytes - this is the ML-5 fix
Glucose	Fasting glucose 1558-6, Random glucose 2345-7, Post-prandial 1521-4, HbA1c 4548-4	mg/dL <-> mmol/L (x 0.0555)
Lipid	Total cholesterol 2093-3, HDL 2085-9, LDL 13457-7, Triglycerides 2571-8	

8. Repository layout (monorepo)

```
mediscan/
|-- apps/
|   |-- api/           Django apps: identity, reports, consultation, billing, administration, audit
|   |   |-- <app>/     models.py services.py policies.py serializers.py views.py tests/
|   |   |-- worker/    Celery app + pipeline/ (one module per step)
|   |   |-- inference/ FastAPI. bundles/ (loader, manifest schema), predictors/, rules/, api/
|   |   |-- llm_gateway/ FastAPI. providers/, rag/, prompts/<name>/<version>.md, validators/
|   |   |-- web/       React + TS. src/features/<domain>/, src/api/ (generated), src/components/
|-- packages/
|   |-- clinical/      Pure Python, no framework deps: analytes.py, units.py, observation.py, ranges.py
|   |   |-- contracts/ openapi/{api,inference,llm}.yaml (committed, CI-checked)
|-- ml/
|   |-- data/          DVC or hash-pinned public datasets. Never PHI.
|   |   |-- pipelines/ train.py --disease X -> dist/<disease>/<version>/
|   |   |               {model, manifest.json, card.md, eval/}
|   |   |-- evaluation/ cv.py, calibration.py, thresholds.py, report_tables.py
|   |   |-- golden_set/ OCR benchmark: pdfs/ (private or consented) + truth/*.json (hand-transcribed)
|-- infra/
|   |-- docker-compose.yml dev: postgres, redis, minio, ollama + all four services + web
|   |-- compose.prod.yml  single-VPS production + caddy
|   |-- scripts/          backup.sh, restore.sh, rotate_keys.md
|-- docs/
|   |-- adr/              0001-cupid.md 0002-async-jobs.md 0003-observation-model.md ...
|   |-- runbook.md        deploy, rollback, incident
|   |-- report/           academic report sources; tables generated from ml/evaluation
|-- .github/workflows/    ci.yml (lint, typecheck, tests, contract-diff, gitleaks, build)
|                           ml-eval.yml (golden-set EMR, CV tables)
|-- .pre-commit-config.yaml ruff, black, eslint, prettier, gitleaks
|-- .env.example
`-- README.md
```

packages/clinical is the heart of the system: zero framework dependencies, exhaustively unit-tested, imported by api, worker and inference. Aliases, units, conversions and reference-range logic live only there.

9. Engineering standards

Every item here is a PR review criterion. A PR that violates one is not merged.

9.1 Contracts first

- DRF (drf-spectacular) and FastAPI emit OpenAPI; specs are committed under packages/contracts and CI fails on drift.
- The web client is generated (openapi-typescript + a thin fetch wrapper). No hand-written URLs in components or slices.
- Service-to-service payloads are pydantic models shared via packages/clinical types.

9.2 Types and lint

- Python: ruff (lint + import sort), black, mypy --strict on packages/clinical and apps/inference; pyright basic elsewhere.
- TypeScript: tsc --strict, eslint (react, hooks, import order), prettier.
- No bare except; no print in application code; no TODO without an issue link.

9.3 Errors, logging, observability

- Structured JSON logs with request_id / job_id propagated across api -> worker -> inference -> llm_gateway.
- Sentry in all services. Prometheus metrics: job duration by step, model latency, OCR field hit-rate, LLM tokens, queue depth.
- /health (liveness) and /ready (dependencies) on every service.
- Fail loudly: contract violations raise; jobs mark FAILED with the step and error; UI shows it.

9.4 Testing - the tests that would have caught Section 3

Test	Catches	Where
Model sensitivity: vary one required analyte, assert probability changes monotonically in the expected direction	ML-1, ML-2 (constant outputs)	apps/inference/tests
Bundle contract: manifest analytes are all producible from a standard CBC/LFT/RFT/TFT/lipid panel; load-time feature-name assertion	ML-1, ML-4, ML-7	apps/inference/tests
Golden-set OCR: field-level exact-match rate on hand-transcribed truth; CI fails if EMR drops below the last release	OCR-1, OCR-2, OCR-3	ml/golden_set + CI job
Rule engine: positive and negative case per rule; property test that evaluation never mutates input	SAF-1, SAF-3	packages/clinical/tests
Unit conversion round-trip and alias resolution tests	ML-5	packages/clinical/tests
Policy matrix: roles x resources x actions, generated from a table	SEC-1, SEC-6, LLM-1	apps/api/*/tests/test_policies.py
Pipeline integration with recorded LLM/inference responses (VCR-style), no network in CI	BE-3, LLM-3	apps/worker/tests
LLM output validator: numbers not in context are rejected	LLM-3	apps/llm_gateway/tests
Payment state machine and idempotent verify (same pidx twice)	PAY-1, PAY-2, PAY-5	apps/api/billing/tests
Playwright e2e: upload -> review -> result -> book -> pay (sandbox)	FE-1, FE-3	apps/web/e2e

9.5 Secrets and configuration

- .env.example committed; .env never. All config via environment; application refuses to start without required keys.
- gitleaks in pre-commit and CI. Rotate Groq, Khalti and Django SECRET_KEY immediately (Phase 0).
- Purge PHI and keys from history with git filter-repo; all collaborators re-clone.

9.6 Data handling

- PHI never leaves the api/worker boundary except as de-identified values to the LLM gateway (no name, ID, DOB, contact).
- Retention policy documented; patient can delete their data (report, observations, explanations, conversations, file).
- Backups nightly; restore tested quarterly; runbook in docs/runbook.md.

9.7 Process

- Trunk-based development; short-lived branches; PR template with 'CUPID check' and 'tests added' sections; one approval required.
- Architecture Decision Records for every decision in Sections 6-7 and any future one that changes a contract.
- Conventional commits; semantic version tags for bundles and services.

10. ML lifecycle

10.1 One pipeline, versioned bundles

```
python -m ml.pipelines.train --disease ckd --config ml/configs/ckd.yaml
-> ml/dist/ckd/2026.09.1/
    model.joblib
    manifest.json      { condition, model_version, algorithm, features:[{loinc, unit, required}],
                      classes, threshold, metrics:{cv_mean, cv_std, test}, dataset_sha256, git_sha,
                      trained_at, selection_rule }
    card.md            dataset, population, intended use, known limitations
    eval/              cv_results.csv, calibration.png, confusion.png, threshold_curve.png
```

Bundles are uploaded to object storage; the inference service pulls a pinned version at start-up. No model artifact is ever committed to git or hand-edited.

10.2 Evaluation protocol

- Stratified 5-fold cross-validation; report mean +- std. A single 80/20 split is not a result.
- Binary tasks: precision, recall, F1, ROC-AUC, PR-AUC, Brier score. Multi-class: macro-F1 plus per-class recall.
- Threshold selected on validation folds for a stated target (e.g. recall ≥ 0.90) and written to the manifest.
- Calibration curve required; probabilities shown to users must be calibrated or labelled as scores.
- One metric definition per table; tables in the report are generated by ml/evaluation/report_tables.py.

10.3 Feature availability policy

Each condition declares which analytes a real Nepali blood panel actually supplies. If a required analyte is missing the assessment is **NOT_ASSESSABLE** and the UI says so. Imputation is allowed only for analytes declared optional in the manifest, and the manifest records the imputation strategy.

10.4 Honest scoping per condition (v1)

Condition	Dataset today	Problem	v1 recommendation
Anemia	Kaggle anemia (Gender, Hb, MCH, MCHC, MCV)	Threshold-derived labels - RF relearns WHO rules	Keep. Present as rule-confirmed classification; also show WHO sex-specific rule result.
CKD	UCI CKD (400 rows)	Small, easy; many features not on a panel	Retrain on sc, bu, hemo, sod, pot, age, sex only; add eGFR (CKD-EPI 2021) as a deterministic companion.
Liver	ILPD (583 rows)	Noisy, F1 ~0.67	Keep with honest probability; ensemble as now; A/G ratio derived.
Thyroid	Garvan 9k rows	27 codes; free vs total confusion	Map to 3 classes deliberately; require TSH + (FT4 or TT4) and use the matching analyte.
Diabetes	PIMA (768 rows, 2h-OGTT glucose, females)	Feature semantics do not match fasting glucose; population mismatch	Reframe as 'glycaemic risk': ADA/WHO rules on fasting glucose and HbA1c first; ML secondary and labelled. Decision D4.
Heart	Kaggle heart failure (918 rows)	Needs ECG/exercise features a blood panel lacks	Remove from v1, or retrain on age/sex/BP/cholesterol/fasting glucose with clearly lower expected performance. Decision D4.

10.5 OCR golden set

- Minimum 30 distinct real reports from at least 8 Nepali labs/hospitals, with written consent or full redaction, stored privately.
- Truth JSON hand-transcribed by one person and verified by a second; disagreements resolved and logged.

- Metrics: field-level EMR (tolerance 1% or 0.01), missed-field rate, hallucinated-field rate, per-lab breakdown. Hallucinations count double.
- Synthetic stress set (OCR-7) reported separately and never called validation.

11. Roadmap and ownership

11.1 Phases

Phase	Duration	Outcome	Key tasks
0 - Stop the bleeding (current code)	1 week	No live data exposure; no constant predictors shipped	Rotate Groq/Khalti/SECRET_KEY; git filter-repo purge of PDFs and keys; fix SEC-1, SEC-2 (disable static media, DEBUG=False), SEC-6, LLM-1; add model sensitivity test and mark Heart/Kidney/Thyroid as unavailable in UI until fixed; remove confidence_score display; delete Booking/, conflicts.txt, status_output.txt, create_doctor.py
1 - Foundations	2-3 weeks	Monorepo with dev environment, CI, contracts, ADRs; nothing user-facing changes	Layout in Section 8; docker-compose (postgres, redis, minio, ollama); CI (lint, typecheck, tests, gitleaks, contract diff); packages/clinical skeleton with Analyte registry and units; OpenAPI committed; TS scaffold + generated client; ADR 0001-0010; PostgreSQL migration; object storage for files
2 - Clinical core	3-4 weeks	Trustworthy extraction -> review -> rules -> assessment pipeline	Observation/Suggestion/CriticalFlag/RiskAssessment models; extraction v2 (TSV rows, deskew, aliases, confidence, bbox); review UI with highlights and suggestions; YAML rule engine + tests; ml/pipelines/train.py + bundles + manifests for Anemia, CKD, Liver, Thyroid (Diabetes/Heart per D4); inference service rewrite; Celery pipeline with ReportJob; result page redesign
3 - Explanation and chat	2 weeks	Grounded, validated, cited explanations; persistent conversations	llm_gateway with provider adapter; prompt versioning; RAG with citations and corpus manifest; output validator; Conversation/Message models; SSE streaming through api; de-identification layer
4 - Consultation and billing	2 weeks	Reliable booking and payment	Consultation state machine; slot validation + DB constraints; single Khalti ePayment v2 flow with lookup + webhook, idempotent; refund handling; audit log wired to all PHI reads; notifications (email) for critical flags and bookings
5 - Scale and ship	2 weeks	Production on one VPS with runbook	k6 load test (target: 50 concurrent uploads, p95 job < 90 s); rate limits; backups + restore drill; compose.prod with Caddy TLS; Sentry + Prometheus + Grafana; runbook; privacy policy + consent; Playwright e2e in CI

11.2 Ownership (four people)

Owner	Areas	Phase 2 deliverable
Frontend	apps/web, generated client, e2e	Review UI with bbox highlights and suggestions; result page with flags / assessed / not-assessable cards; job progress
ML	ml/, apps/inference, packages/clinical (co-owner)	train.py, bundles + manifests for 4-6 conditions, inference service, sensitivity + contract tests, golden-set harness
Backend	apps/api, apps/worker, packages/clinical (co-owner), infra	Domain models, services, policies, Celery pipeline, PostgreSQL + MinIO, CI
LLM / QA	apps/llm_gateway, rules YAML, testing, docs	Gateway + validator + RAG citations; rule catalogue with guideline refs; policy matrix tests; ADRs and runbook

packages/clinical is jointly owned by ML and Backend; changes require both reviewers.

12. Open decisions

These must be closed (and recorded as ADRs) before Phase 1 starts. Recommendation in bold.

ID	Decision	Options	Recommendation and reasoning
D1	Inference service and training pipeline	Rewrite / refactor	Rewrite. ~2,200 lines whose core abstractions caused most S1 findings.
D2	Core domain type	Observation + LOINC (FHIR-lite) / keep dict of keys	Observation + LOINC. Fixes unit and alias bugs structurally; enables HMS/FHIR integration.
D3	Missing features	NOT_ASSESSABLE / impute silently	NOT_ASSESSABLE. Honest, defensible, and the UI can explain what to test next.
D4	Heart and Diabetes in v1	Drop / retrain on available features / keep as-is with warning	Heart: drop or retrain on blood-panel features. Diabetes: rules-first (fasting glucose, HbA1c) with ML secondary.
D5	LLM default provider and privacy stance	Local Ollama / cloud with de-identification / cloud raw	Cloud with de-identified payload for v1; local as config flag. Never send name, ID, DOB, contact.
D6	Job queue	Celery + Redis / Dramatiq / Django-Q / Postgres-backed	Celery + Redis. Standard, Redis already a dependency, best tooling.
D7	Frontend language	TypeScript now / JS + generated client only	TypeScript now, feature by feature; the generated client is TS anyway.
D8	Deployment target v1	Single VPS + Compose / managed PaaS / Kubernetes	Single VPS + Compose. Stateless services keep the k8s path open.
D9	OCR engine	Tesseract TSV / PaddleOCR / cloud OCR	Tesseract TSV first (CPU, offline, known); evaluate PaddleOCR on the golden set in Phase 2 and switch if EMR gain > 10 points.
D10	Golden-set data governance	Consent forms / full redaction / synthetic only	Consent + redaction, stored privately; synthetic only for stress tests.
D11	Academic report	Correct in place / rewrite results chapter	Rewrite results chapter from generated tables once Phase 2 bundles exist.

13. Definition of done and checklists

13.1 Pull request checklist

- Reads in domain language (Section 7); no new dict-shaped data crossing a service boundary.
- One responsibility per module/function; no step both computes and performs I/O without a reason stated.
- Inputs validated; failures raise or return typed errors; no silent fallback.
- Tests added for behaviour, including at least one negative case; policy matrix updated if a permission changed.
- OpenAPI regenerated if an endpoint changed; client regenerated; no hand-written URL.
- Logs are structured and carry request/job id; no print.
- No secret, PHI or model artifact in the diff.
- ADR added or updated if a Section 6-7 decision changed.

13.2 Release checklist

- CI green including golden-set EMR gate and sensitivity tests.
- Bundle versions pinned and manifests reviewed; model cards updated.
- Migrations applied on staging; backup taken before deploy; rollback tested.
- Sentry clean for 24 h on staging; p95 job duration within target.
- Runbook updated; release notes list clinical-facing changes explicitly.

13.3 Phase 0 checklist (start today)

- 1 Rotate Groq API key, Khalti secret, Django SECRET_KEY; move all to .env; add .env.example.
- 2 git filter-repo: remove Backend/reports/*.pdf, Backend/doctor_licenses/*, ML/README.md key lines; force-push; team re-clones.
- 3 settings.py: DEBUG default False; remove static media serving; ALLOWED_HOSTS from env.
- 4 Add policy checks to DoctorAvailabilityViewSet writes, trends endpoint, chatbot context lookup.
- 5 Add tests/test_model_sensitivity.py; disable Heart, Kidney, Thyroid assessments in analyze_full_report until bundles exist; show 'temporarily unavailable'.
- 6 Remove confidence_score from the result page; persist critical_alerts on ReportResult and render a banner.
- 7 Change sanitize_data to return suggestions (do not overwrite) - or disable it - until the review UI supports suggestions.
- 8 Delete Booking/, conflicts.txt, status_output.txt, create_doctor.py, duplicate SampleReport files; replace root README.
- 9 Add pre-commit with ruff, black, eslint, prettier, gitleaks; add minimal CI running existing tests.
- 10 Write ADR 0001 (CUPID) and ADR 0002 (this baseline) into docs/adr/.

Appendix A - Evidence: commands and observed outputs

All results reproduced on 18 Sep 2026 against main @ 5043aa8 using the repository's venv.

A.1 Heart feature misalignment

```
>>> sc = joblib.load('models/heart_scaler.pkl'); used = joblib.load('models/heart_model_columns.pkl')
>>> new = joblib.load('models/heart_columns.pkl')
inference thinks 'Sex' | scaler was fit on 'RestingBP' mean=132.40
inference thinks 'RestingBP' | scaler was fit on 'Cholesterol' mean=198.80
inference thinks 'Cholesterol' | scaler was fit on 'FastingBS' mean=0.23
inference thinks 'MaxHR' | scaler was fit on 'Oldpeak' mean=0.89
>>> for ch in [120,172,250,350]: engine.predict_heart({'Cholesterol': ch})
{'prediction': 'Heart Disease', 'risk_score': 67.05} # x4, identical
```

A.2 Kidney constant output

```
>>> joblib.load('models/kidney_columns.pkl')
['sc', 'bu', 'hemo', 'sod', 'pot', 'sg', 'al', 'su', 'age', 'bp', 'bgr', 'htn', 'dm']
>>> type(joblib.load('models/kidney_best_model.pkl')).__name__
'LogisticRegression'
>>> engine.predict_kidney({'Creatinine':1.5, 'Blood_Urea':20, 'Hemoglobin':15}) -> CKD 50.89
>>> engine.predict_kidney({'Creatinine':4.9, 'Blood_Urea':150, 'Hemoglobin':7}) -> CKD 50.89
```

A.3 Thyroid classes

```
>>> joblib.load('models/thyroid_best_model.pkl').classes_
array([ 0, 1, 2, 3, 4, 5, 6, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 28, 29, 30, 31])
>>> pd.read_csv('datasets/thyroid_big.csv')['target'].value_counts().head(6)
'-': 6771, 'K': 436, 'G': 359, 'I': 346, 'F': 233, 'R': 196
```

A.4 Safety sanitiser masking criticals

```
raw={'Creatinine': 16.0} sanitized={'Creatinine': 1.6} alerts_after=[]
raw={'Bilirubin_Total': 25.0} sanitized={'Bilirubin_Total': 2.5} alerts_after=[]
raw={'TSH': 150.0} sanitized={'TSH': 15.0} alerts_after=[]
```

A.5 OCR on the real scans

```
Report1.pdf (Bir Hospital) 26 fields. MCH=34.0 (document: 29). Total_Protiens=6.0 (document: 6.9)
Report4.pdf (Civil Service) 9 fields. Creatinine=9.0 (document: 0.9). Hemoglobin/WBC/RBC/Sodium missed.
Alamine_Aminotransferase=4.0 (not present on the document)
md5: Report5 = Report10 = Report1; Report6 = Report2; Report7 = Report9 = Report3; Report8 = Report4
```

A.6 Secrets and PHI in git

```
$ git log --all --oneline -S'gsk_' --name-only
bd83850 Complete AI with Docker and Optimized Models ML/README.md
4df8c74 Add Hybrid LLM, Safety Guard, and ignore ... ML/README.md
$ git ls-files | grep -E '\.pdf$'
Backend/doctor_licenses/license.pdf Backend/reports/Report1.pdf Backend/reports/Report2.pdf Backend/reports/Report3.
$ grep -n "Key df19" Backend/doctor/services.py
13: "Authorization": f"Key df19d96325c548c09fdf0bf2aaf684b3",
```

A.7 Report table provenance

```
benchmark_results.csv (weighted averages): Heart LR 0.8913/0.8922/0.8913/0.8908 RF 0.8967/0.8968/0.8967/0.8965
Report Figure 17: Heart LR 0.8913/0.8922/0.8913/0.8908 RF 0.8967/0.8968/0.8967/0.8965
SVM 0.9000/0.8900/0.9412/0.9143 <- binary metrics, different run
```

Appendix B - What the project does well (keep these)

- **Human-in-the-loop OCR** with raw/final/is_corrected audit trail - becomes Observation supersession in the new model.
- **Recall-first thresholding with reasoning** - becomes manifest.threshold chosen on validation folds.
- **Role-aware summaries** and serializer-level hiding - becomes Explanation.audience with policies.
- **Specialist routing** into doctor search and booking - keep as a consultation service function.
- **Authentication**: OTP per login, HttpOnly cookies, OAuth blocked for doctors, licence verification state machine - keep as identity app.
- **Trends** endpoint and chart - becomes a first-class Observation time series per analyte.
- **Streaming chat** plumbing - keep the transport, add persistence and validation.
- **Revenue split and refund policy** - keep the business rules, move to billing service with a real state machine.
- **Docker multi-stage ML image** - template for all four services.
- **Honest tone about weak models** (Liver, Diabetes) and t-SNE reasoning in the report - keep it, extend it to everything.

End of document.